JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent.atomic

Class AtomicLongArray

java.lang.Object
    java.util.concurrent.atomic.AtomicLongArray

All Implemented Interfaces:

Serializable

```
public class AtomicLongArray
extends Object
implements Serializable
```

A `long` array in which elements may be updated atomically. See the `java.util.concurrent.atomic` package specification for description of the properties of atomic variables.

Since:

1.5

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`AtomicLongArray(int length)`	Creates a new AtomicLongArray of the given length, with all elements initially zero.
`AtomicLongArray(long[] array)`	

	Creates a new AtomicLongArray with the same length as, and all elements copied from, the given array.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
long	**accumulateAndGet**(int i, long x, **LongBinaryOperator** accumulatorFunction)	Atomically updates the element at index i with the results of applying the given function to the current and given values, returning the updated value.
long	**addAndGet**(int i, long delta)	Atomically adds the given value to the element at index i.
boolean	**compareAndSet**(int i, long expect, long update)	Atomically sets the element at position i to the given updated value if the current value == the expected value.
long	**decrementAndGet**(int i)	Atomically decrements by one the element at index i.
long	**get**(int i)	

		Gets the current value at position i.
long	**getAndAccumulate**(int i, long x, **LongBinaryOperator** accumulatorFunction)	Atomically updates the element at index i with the results of applying the given function to the current and given values, returning the previous value.
long	**getAndAdd**(int i, long delta)	Atomically adds the given value to the element at index i.
long	**getAndDecrement**(int i)	Atomically decrements by one the element at index i.
long	**getAndIncrement**(int i)	Atomically increments by one the element at index i.
long	**getAndSet**(int i, long newValue)	Atomically sets the element at position i to the given value and returns the old value.
long	**getAndUpdate**(int i, **LongUnaryOperator** updateFunction)	Atomically updates the element at index i with

		the results of applying the given function, returning the previous value.
long	**incrementAndGet**(int i)	Atomically increments by one the element at index i.
void	**lazySet**(int i, long newValue)	Eventually sets the element at position i to the given value.
int	**length**()	Returns the length of the array.
void	**set**(int i, long newValue)	Sets the element at position i to the given value.
String	**toString**()	Returns the String representation of the current values of array.
long	**updateAndGet**(int i, **LongUnaryOperator** updateFunction)	Atomically updates the element at index i with the results of applying the given function, returning the updated value.

boolean	**weakCompareAndSet**(int i, long expect, long update)	Atomically sets the element at position i to the given updated value if the current value == the expected value.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

AtomicLongArray

```
public AtomicLongArray(int length)
```

Creates a new AtomicLongArray of the given length, with all elements initially zero.

Parameters:

`length - the length of the array`

AtomicLongArray

```
public AtomicLongArray(long[] array)
```

Creates a new AtomicLongArray with the same length as, and all elements copied from, the given array.

Parameters:

`array - the array to copy elements from`

Throws:

NullPointerException - if array is null

Method Detail

length

```
public final int length()
```

Returns the length of the array.

Returns:

`the length of the array`

get

```
public final long get(int i)
```

Gets the current value at position `i`.

Parameters:

`i - the index`

Returns:

`the current value`

set

```
public final void set(int i,
                      long newValue)
```

Sets the element at position `i` to the given value.

Parameters:

`i - the index`

`newValue - the new value`

lazySet

```
public final void lazySet(int i,
                          long newValue)
```

Eventually sets the element at position `i` to the given value.

Parameters:

`i - the index`

`newValue - the new value`

Since:

1.6

getAndSet

```
public final long getAndSet(int i,
                            long newValue)
```

Atomically sets the element at position `i` to the given value and returns the old value.

Parameters:

`i - the index`

`newValue - the new value`

Returns:

`the previous value`

compareAndSet

```
public final boolean compareAndSet(int i,
                                   long expect,
                                   long update)
```

Atomically sets the element at position `i` to the given updated value if the current value == the expected value.

Parameters:

`i - the index`

`expect - the expected value`

`update - the new value`

Returns:

`true if successful. False return indicates that the actual value was not equal to the expected value.`

weakCompareAndSet

```
public final boolean weakCompareAndSet(int i,
                                       long expect,
                                       long update)
```

Atomically sets the element at position `i` to the given updated value if the current value == the expected value.

May fail spuriously and does not provide ordering guarantees, so is only rarely an appropriate alternative to `compareAndSet`.

Parameters:

`i - the index`

`expect - the expected value`

`update - the new value`

Returns:

`true if successful`

getAndIncrement

```
public final long getAndIncrement(int i)
```

Atomically increments by one the element at index `i`.

Parameters:

`i - the index`

Returns:

`the previous value`

getAndDecrement

```
public final long getAndDecrement(int i)
```

Atomically decrements by one the element at index `i`.

Parameters:

`i - the index`

Returns:

`the previous value`

getAndAdd

```
public final long getAndAdd(int i,
                            long delta)
```

Atomically adds the given value to the element at index `i`.

Parameters:

`i - the index`

`delta - the value to add`

Returns:

`the previous value`

incrementAndGet

```
public final long incrementAndGet(int i)
```

Atomically increments by one the element at index `i`.

Parameters:

`i - the index`

Returns:

`the updated value`

decrementAndGet

```
public final long decrementAndGet(int i)
```

Atomically decrements by one the element at index `i`.

Parameters:

`i - the index`

Returns:

`the updated value`

addAndGet

```
public long addAndGet(int i,
                      long delta)
```

Atomically adds the given value to the element at index `i`.

Parameters:

`i - the index`

`delta - the value to add`

Returns:

`the updated value`

getAndUpdate

```
public final long getAndUpdate(int i,
                               LongUnaryOperator updateFunction)
```

Atomically updates the element at index `i` with the results of applying the given function, returning the previous value. The function should be side-

effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`i - the index`

`updateFunction - a side-effect-free function`

Returns:

`the previous value`

Since:

`1.8`

updateAndGet

```
public final long updateAndGet(int i,
                               LongUnaryOperator updateFunction)
```

Atomically updates the element at index `i` with the results of applying the given function, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`i - the index`

`updateFunction - a side-effect-free function`

Returns:

`the updated value`

Since:

`1.8`

getAndAccumulate

```
public final long getAndAccumulate(int i,
                                   long x,
                                   LongBinaryOperator accumulatorFunction)
```

Atomically updates the element at index `i` with the results of applying the given function to the current and given values, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value at index `i` as its first argument, and the given update as the second argument.

Parameters:

`i - the index`

x - the update value

accumulatorFunction - a side-effect-free function of two arguments

Returns:

the previous value

Since:

1.8

accumulateAndGet

```
public final long accumulateAndGet(int i,
                                   long x,
                                   LongBinaryOperator accumulatorFunction)
```

Atomically updates the element at index `i` with the results of applying the given function to the current and given values, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value at index `i` as its first argument, and the given update as the second argument.

Parameters:

i - the index

x - the update value

accumulatorFunction - a side-effect-free function of two arguments

Returns:

the updated value

Since:

1.8

toString

```
public String toString()
```

Returns the String representation of the current values of array.

Overrides:

toString in class Object

Returns:

the String representation of the current values of array

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.